NoteVault Secure Build Report

Threat model, vulnerability findings, remediation and supply-chain controls

Team: [TEAM_ID]
Submission deadline: 6 November 2026, 16:59 ICT

GitHub repository
[Add repository URL]

Demonstration video
[Add unlisted YouTube or link-accessible Drive URL]

Executive summary

NoteVault is a deliberately scoped Flask notes application. This report describes the assets and trust boundaries, the security findings in the vulnerable baseline, the remediations in the secure build, and reproducible supply-chain controls. Only run attack demonstrations against the local lab instance.

1. Threat model

Scope and assumptions

Local web app on port 8080; SQLite persistence; a browser is untrusted; reverse proxy/TLS termination is outside the demo. Users may be malicious. Production secrets and public deployment are out of scope.

Data-flow diagram

```
flowchart LR
 B[Browser: untrusted] -->|HTTPS request, cookie, form| W[Flask web app]
 W -->|parameterized SQL| D[(SQLite notes and password hashes)]
 W -->|PDF export| E[ReportLab in process]
 W -->|HTML response, Jinja autoescape| B
 A[Admin browser] -->|authenticated request| W
 subgraph TB1[Trust boundary: network / browser to app]
 B
 end
 subgraph TB2[Trust boundary: app process to persistent storage]
 W
 D
 end
```

STRIDE analysis

Threat: Spoofing | **Component / boundary**: Login boundary | **Example**: Credential guessing or stolen cookie | **Control**: Strong password hashes, secure cookie flags, rotate session at login

Threat: Tampering | **Component / boundary**: Forms to app | **Example**: Cross-site note creation or forged fields | **Control**: CSRF token, server-side validation

Threat: Repudiation | **Component / boundary**: App and audit trail | **Example**: User denies changing a note | **Control**: Authenticated owner_id; production audit logging remains recommended

Threat: Information disclosure | **Component / boundary**: Notes / export | **Example**: IDOR exposes another user's note | **Control**: Owner-scoped queries, admin gate, no secret in source

Threat: Denial of service | **Component / boundary**: Request / export | **Example**: Oversized content or expensive repeated requests | **Control**: 1 MiB request cap, field bounds; production rate limit still required

Threat: Elevation of privilege | **Component / boundary**: User to admin | **Example**: Role supplied by registration or stale session | **Control**: Role assigned server-side, decorator authorization

2. Vulnerability report

Severity uses CVSS-like qualitative ratings for this teaching app; exploitability is local and assumes an authenticated or reachable instance.

ID: F-01 | **Finding and severity**: SQL injection in search — High | **Reproduction in vulnerable baseline**: Open `/search?q=%25'%20OR%201%3D1%20--%20` in baseline; SQL is assembled with string formatting | **CWE**: CWE-89 | **OWASP Top 10:2025**: A05 Injection | **Impact**: Reads notes outside the intended search predicate

ID: F-02 | **Finding and severity**: Broken access control / IDOR — High | **Reproduction in vulnerable baseline**: Change `/export/1` to another user's note id | **CWE**: CWE-639 | **OWASP Top 10:2025**: A01 Broken Access Control | **Impact**: Unauthorized note disclosure

ID: F-03 | **Finding and severity**: Stored XSS — High | **Reproduction in vulnerable baseline**: POST an `<img src=x onerror=alert(1)>` title/body to `/note`, then open `/note/1` | **CWE**: CWE-79 | **OWASP Top 10:2025**: A05 Injection | **Impact**: Script executes in readers' sessions

ID: F-04 | **Finding and severity**: Weak password storage / demo default — High | **Reproduction in vulnerable baseline**: Inspect SQLite hash/plaintext or log in with seeded admin credentials | **CWE**: CWE-256, CWE-521 | **OWASP Top 10:2025**: A07 Authentication Failures | **Impact**: Account takeover and administrator compromise

ID: F-05 | **Finding and severity**: CSRF on state changes — Medium | **Reproduction in vulnerable baseline**: POST `/note` has no CSRF token; submit a forged cross-origin request while logged in | **CWE**: CWE-352 | **OWASP Top 10:2025**: A01 Broken Access Control | **Impact**: Forged actions using victim's session

ID: F-06 | **Finding and severity**: Session fixation and weak cookie policy — Medium | **Reproduction in vulnerable baseline**: Inspect static signing key and missing SameSite/Secure flags; session identity is not regenerated after login | **CWE**: CWE-384, CWE-1004 | **OWASP Top 10:2025**: A07 Authentication Failures | **Impact**: Session theft or fixation

ID: F-07 | **Finding and severity**: Dependency vulnerabilities / obsolete pins — High | **Reproduction in vulnerable baseline**: `pip-audit -r needs-vulnerable.txt`; review affected transitive dependencies | **CWE**: CWE-1104 | **OWASP Top 10:2025**: A03 Software Supply Chain Failures | **Impact**: Known public CVEs can enable compromise

ID: F-08 | **Finding and severity**: Insecure container build — High | **Reproduction in vulnerable baseline**: Inspect baseline Dockerfile for an end-of-life Python base, root user and broad build context | **CWE**: CWE-250, CWE-1104 | **OWASP Top 10:2025**: A02 Security Misconfiguration | **Impact**: Known runtime flaws and unnecessary container privileges increase impact

3. Remediation mapping

Finding: F-01 | **Secure implementation**: Parameterized SQLite queries; wildcard input bounded

Finding: F-02 | **Secure implementation**: Owner-scoped read/export; admin decorator checks server-side role

Finding: F-03 | **Secure implementation**: Jinja autoescaping; no string-built HTML

Finding: F-04 | **Secure implementation**: Werkzeug PBKDF2 password hashes; registration requires 12+ chars; no default production password

Finding: F-05 | **Secure implementation**: Per-session CSRF token on all state-changing forms

Finding: F-06 | **Secure implementation**: HttpOnly and SameSite=Lax; session cleared and rotated at login; Secure enabled behind HTTPS

Finding: F-07 | **Secure implementation**: Current pinned dependencies, pip-audit failure gate, CycloneDX SBOM

Finding: F-08 | **Secure implementation**: Pinned slim base, non-root UID, read-only root FS, dropped capabilities, no-new-privileges, no baked secrets

4. Secure build and supply chain

• SBOM: `sbom.cdx.json` in CycloneDX JSON format generated from `needs.txt`.

• Container: multi-stage build is unnecessary for this small app; the runtime image uses a non-root account and only copies runtime files.

• Signing: GitHub Actions requests short-lived Sigstore identity via OIDC; signs the image archive and attests the SBOM. Verify with `cosign verify-blob` and `cosign verify-attestation-blob` using the workflow identity and issuer constraints.

• Provenance: GitHub workflow run, commit SHA, and artifact bundles are uploaded with the build. Protect `main` and require successful `scans` before merge.

5. Security CI

GitHub Actions runs Bandit (SAST), pip-audit (SCA), and Gitleaks secret scanning on pushes and pull requests. Any Bandit medium/high finding (`-ll`), any pip-audit vulnerability, or detected secret fails the job. Main-branch build creates SBOM, builds and signs artifact material.

6. Attack, root cause, fix demonstration

For each finding: show the vulnerable behavior in the baseline, identify the unsafe data flow or missing authorization check, switch to the secure implementation, replay the same request, and show the denied/escaped/isolated outcome. Do not include real credentials or expose the app publicly.

7. Limitations and residual risks

This classroom app has no production rate limiting, password reset, email verification, audit log, backup encryption, TLS proxy configuration, or formal authorization review. Set `COOKIE_SECURE=1` behind HTTPS, inject random secrets from a secret manager, and review dependency/image scans at release time.

8. Individual contribution (submit privately)

Member: [Name] | **Contribution**: [specific work] | **Assessment**: [Full / Most / Some / Little + reason]

Member: [Name] | **Contribution**: [specific work] | **Assessment**: [Full / Most / Some / Little + reason]

Appendix: run locally

Set `TEAM_ID` once and retain it. Set strong `SECRET_KEY` and `ADMIN_PASSWORD`, then run `docker compose up --build` from `project/starter-app`. Demo credentials are not published; create a user through registration. This PDF is a starter report; replace bracketed team/repo/video fields and enter truthful contribution statements before submission.

References

- OWASP Top 10 2025
- OWASP A01 Broken Access Control
- OWASP A05 Injection